Answer the following questions.

1. A class `Rectangle` has no user-defined constructor. Which statement is true about constructing a `Rectangle` object?
   a) `Rectangle r;` will cause a compilation error because no constructor is defined.
   b) `Rectangle r;` will compile but always crash at runtime.
   c) The compiler generates a default constructor automatically since no constructor is defined.
   d) A constructor with no parameters must always be written explicitly.

   answer: c

2. Consider the following C++ class definition:

```
class Timer {
    int seconds;

public:
    Timer(int s) : seconds(s) {}

    ~Timer() {
        cout << "Timer stopped\n";
    }
};
```

   When is the destructor called for a `Timer` object created on the stack?
   a) When the programmer explicitly calls delete on it.
   b) When the object goes out of scope.
   c) When the program starts.
   d) Only when the garbage collector runs.

   answer: b

3. A class `Shape` defines the constructor `Shape(float x, float y)`. Which statement is correct?
   a) `Shape s;` is valid
   b) `Shape s;` is invalid
   c) `Shape s(1.0, 2.0);` is invalid.
   d) The compiler generates a default constructor in addition to the user-defined constructor.

   answer: b

4. Which of the following statements uses copy initialization to create an object of type `Color`?

```
a) Color c{255, 0, 0};
b) Color c(255, 0, 0);
c) Color c2 = c1;
d) Color c;
```

   answer: c

5. Assume `Point` is a class containing two data members `int x;` and `int y;` and has no user-defined constructor.

   Given the declaration:
   ```
   Point p{};
   ```

   What are the values of `p.x` and `p.y` after initialization?
   a) The values are undefined because brace initialization does not initialise members.
   b) Both x and y contain indeterminate (garbage) values because no constructor is defined.
   c) This results in a compilation error because `Point` cannot be initialised using empty braces.
   d) Both x and y are initialised to 0 due to value-initialisation.

   answer: d

6. Consider the following program:

```
#include <iostream>
using namespace std;
class Dog {
public:
    Dog() { cout << "Default\n"; }
    Dog(const Dog&) { cout << "Copy\n"; }
};
int main() {
    Dog d1;
    Dog d2(d1);
    Dog d3 = d1;
}
```

   What is the output of the program?
   a) Default Copy
   b) Default Default Copy
   c) Default Copy Copy
   d) Default Default Default

   answer: c

7. Which feature can a C++ `struct` have that a C `struct` cannot?
   a) Multiple data members of different types
   b) Member functions with access specifiers (e.g., public/private)
   c) Only global-scope nested type declarations
   d) Variables of the struct type

   answer: b

8. You insert the values {10, 5, 10, 3, 7} into a `std::set<int>`. What does the set contain, and in what order?
   a) {10, 5, 10, 3, 7}, insertion order is preserved
   b) {3, 5, 7, 10}, duplicates are removed and elements are stored in ascending order
   c) {10, 5, 3, 7}, duplicates are removed but insertion order is preserved
   d) {10, 10, 7, 5, 3}, elements are stored in descending order with duplicates

   answer: b

9. Consider the following C++ code segment:

```
class Buffer {
public:
    int* data;

    Buffer(int size) {
        data = new int[size];
    }

    ~Buffer() {
        delete[] data;
    }
};

Buffer b1(5);
Buffer b2 = b1;

*b1.data = 99;
```

What problem occurs when `b1` and `b2` are destroyed?

a) No problem; each object owns separate dynamically allocated memory.
b) `b2.data` is automatically set to `nullptr`, so only `b1` deallocates memory.
c) Double deletion occurs because both objects point to the same memory
d) A compile-time error occurs because the class defines a destructor.

answer: c

10. Consider the following C++ code:

```
int* p = new int(42);

int* q = (int*)malloc(sizeof(int));
```

Which statement correctly distinguishes `new` from `malloc` in this context?

a) `new` requires an explicit cast, whereas `malloc` does not.
b) `malloc` is an operator, whereas `new` is a function.
c) Both behave identically for primitive types like `int`.
d) `new` allocates memory and initialises the object (for non-primitive types it calls constructors), while `malloc` only allocates raw memory and does not initialise or call constructors.

answer: d

11. Which statement about the `this` pointer in C++ is correct?

a) The `this` pointer can be used inside static member functions
b) The `this` pointer refers to the current object that invoked the member function
c) The `this` pointer can be reassigned to refer to another object
d) Every class contains only one shared this pointer for all objects

answer: b

12. Consider the following function template:

```
template<typename T>
T square(T x) {
    return x * x;
}
```

At what point does the compiler generate machine code for square<double>?

a) When the template is defined
b) When the file containing the template is compiled
c) When `square<double>` is first instantiated in the program
d) At program startup before `main()` runs

answer: c

13. Consider the following program:

```
#include <iostream>
using namespace std;

template<typename T>
void display(T val) {
    cout << "Value: " << val << endl;
}

int main() {
    display(3.14);
    display('Z');
}
```

What is the output of the program?

A. Compilation error
B. Value: 3.14
   Value: Z
C. Value: 3
   Value: 90
D. Garbage output

answer: b

14. Which statement is true about the `this` pointer?

a) It stores the address of the class
b) It stores the address of the current object
c) It is a global pointer shared by all objects
d) It is only used in static functions

answer: b

15. Consider the following code:

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<string, int> scores;

    scores["Alice"] = 90;
    scores["Bob"] = 85;

    cout << scores.at("Carol");

    return 0;
}
```

What happens at runtime?

a) Prints 0, `at()` value-initialises missing keys
b) Prints garbage value
c) Throws `std::out_of_range` because `at()` performs bounds checking
d) Inserts `"Carol"` with value 0 silently

answer: c

16. Consider the following code:

```
class Matrix {

    int data[3][3];

public:

    friend Matrix multiply(Matrix, Matrix);

};
```

What is true about the function `multiply`?

a) `multiply` is a member function of `Matrix`
b) `multiply` can access the private data member `data` of `Matrix`
c) `multiply` must be declared in the public section of `Matrix`
d) `multiply` automatically becomes a member of all classes that inherit from `Matrix`

answer: b

17. State True or False: For a `std::set<int>` named `nums`, calling `nums.count(42)` can return a value greater than 1.

answer: false

18. Consider the following code:

```
class Engine;

class Car {
    int horsepower;

public:
    Car(int hp) : horsepower(hp) {}

    friend class Engine;
};
```

Which statement is correct?

a) Only the constructor of `Engine` can access `Car::horsepower`
b) All member functions of `Engine` can access the private member `horsepower` of `Car`
c) `Car` automatically becomes a friend of `Engine`
d) Only static member functions of `Engine` can access Car's private members

answer: b

19. Consider the following code:

```
double x = 5.5;
double& ref = x;
ref *= 2;
cout << x << " " << ref;
```

What is the output of the program?

a) 5.5 11
b) 11 11
c) 5.5 5.5
d) 11 5.5

answer: b

20. Which of the following function templates correctly swaps two variables of any type?

a) `template<class T> void swapVals(T a, T b) { T tmp=a; a=b; b=tmp; }`
b) `template<class T> void swapVals(T& a, T& b) { T tmp=a; a=b; b=tmp; }`
c) `template<class T> void swapVals(T* a, T* b) { T tmp=*a; *a=*b; *b=tmp; } called as swapVals(x, y);`
d) `void swapVals(auto& a, auto& b) { auto tmp=a; a=b; b=tmp; }`

answer: b

21. Consider the following C++ code:

```
class Sensor {
    double reading;

public:
    void setReading(double reading) {
        _______ = reading;  // assign parameter to member
    }
};
```

What correctly fills the blank?

a) Reading
b) Sensor::reading
c) this->reading
d) *reading

answer: c

22. Given the following declaration of class Foo, which of the following are valid ways to construct an object of class Foo? (Note the difference between parentheses and curly braces).

```
class Foo {
public:
    int i = 0;
    Foo(int x) : i(x) {}
};
a) Foo f;
b) Foo f{};
c) Foo f(1);
d) Foo f(Foo(1));
```

answer: c and d

23. For a class template `Pair<T>` with two data members `first` and `second` of type T, which syntax correctly defines the member function `getMax()` outside the class template body?

```
a) T Pair::getMax() { return first > second ? first : second; }
b) template<class T> T Pair<T>::getMax() { return first > second ?
   first : second; }
c) template<class T> T getMax() { return first > second ? first :
   second; }
d) T Pair<T>::getMax() { return first > second ? first : second; }
```

answer: b